Week 8 APP DEV-2

2024 Dec22 ET

1. [Source Q7; MCQ]

In the following code, which attributes ensure that the cookie will only be sent over HTTPS and restrict it from cross-site requests?

```
response.set_cookie('session_id', 'abc123', secure=option1, samesite=option2)
```

(a) secure=True and samesite='Strict'

(b) secure=False and samesite='Strict'

(c) secure=True and samesite='None'

(d) secure=False and samesite='Lax'

[Source ans: a]

2. [Source Q11; MCQ]

Suppose you want the user data to only be accessible by an authenticated user. What security issue exists in this code?

// javascript

```
methods: {
  async fetchUserData() {
    const response = await fetch(`/api/user/${this.userId}`)
    const data = await response.json()
    this.userData = data
  }
}
```

Flask route

```
@app.route('/api/user/<user_id>')
def get_user(user_id):
    query = f"SELECT * FROM users WHERE id = {user_id}"
    result = db.execute(query)
    return jsonify(result)
```

(a) No authentication check

(b) SQL injection vulnerability

(c) Unvalidated user input

(d) All of these

[Source ans: d]

3. [Source Q23; MCQ]

What will happen if a page includes this CSP header and the script attempts to load an external JavaScript file from `otherurl.com`?

CSP Header:

```
Content-Security-Policy: default-src 'self';
```

HTML :

```
<script src="https://otherurl.com/code.js"></script>
```

(a) The script from `otherurl.com` will load normally.

(b) The script will fail to load, and an error will be logged in the console.

(c) The browser will display a warning and allow the script to load.

(d) Nothing will happen; the CSP only affects inline scripts.

[Source ans: b]

2025 Apr13 ET

4. [Source Q18; MCQ]

Which of the following options is the correct sequence of steps in JWT authentication?

(a) Client sends credentials, server validates and responds with JWT, client storesJWT, client sends JWT with each request.

(b) Client sends credentials, server validates and stores session ID, client sendssession ID with each request.

(c) Client sends JWT, server validates and issues a new JWT for each request.

(d) Client sends JWT, server decrypts JWT and checks credentials again.

[Source ans: a]

5. [Source Q25; MCQ]

Consider the below flask implementation with JWT based authentication.

```
from flask import Flask, request, jsonify
import jwt
import datetime

app = Flask(__name__)
SECRET_KEY = "supersecret"

def generate_token(user):
      return jwt.encode(
      {"user": user, "exp": datetime.datetime.utcnow() +
datetime.timedelta(seconds=10)},
      SECRET_KEY,
      algorithm="HS256"
      )

@app.route("/login", methods=["POST"])
def login():
      user = request.json.get("user")
      token = generate_token(user)
      return jsonify({"token": token})

@app.route("/protected", methods=["GET"])
def protected():
      token = request.headers.get("Authorization").split(" ")[1]
      try:
      payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
      return jsonify({"message": f"Hello {payload['user']}!"})
      except jwt.ExpiredSignatureError:
      return jsonify({"error": "Token expired"}), 401
      except jwt.InvalidTokenError:
```

```
        return jsonify({"error": "Invalid token"}), 403
```

If a Vue.js client sends this request 15 seconds after login, what response will it receive (assume the token sent is the one, which was received from login)?

```
fetch("http://localhost:5000/protected", {
  method: "GET",
  headers: { "Authorization": "Bearer <JWT_TOKEN>" }
})
.then(res => res.json())
.then(console.log);
```

(a) { "message": "Hello user!" }

(b) { "error": "Token expired" }

(c) { "error": "Invalid token" }

(d) { "message": "Access granted" }

[Source ans: b]

6. [Source Q27; MSQ]

Which of the following are advantages of using GraphQL over REST?

(a) It allows clients to fetch only the required fields, reducing over-fetching.

(b) It replaces the need for authentication mechanisms.

(c) It improves database performance automatically.

(d) It enforces a strict API response format.

[Source ans: a]

Shared context (source Q30)

Based on the above data, answer the given subquestions.

Consider the following flask app implemented with JWT. The application running on `http://127.0.0.1:5000` and the user logs in using the `/login` endpoint.

Read the following questions and select the correct statement.

```
app = Flask(__name__)
app.config["JWT_SECRET_KEY"] = "supersecretkey"
jwt = JWTManager(app)

@app.route("/login", methods=["POST"])
def login():
    user = {"username": "student", "role": "student"}
    access_token = create_access_token(identity=user)
    return jsonify(access_token=access_token)

@app.route("/protected", methods=["GET"])
@jwt_required()
def protected():
    current_user = get_jwt_identity()
    if current_user.get("role") != "admin":
        return jsonify({"message": "Forbidden: Insufficient permissions"}),
403

    return jsonify(message=f"Hello, {current_user['username']}!")

app.run()
```

7. [Source Q31; MCQ]

What will be the response by the following fetch request.

```
fetch("http://127.0.0.1:5000/protected", {
        method: "GET"
    })
    .then(response => {
        return response.json();
    })
    .then(data => console.log("Protected Route Response:", data))
    .catch(error => console.error("Error:", error));
```

(a)

The request will be processed successfully, and "Hello, student!" will be returned.

(b) The request will fail with a `401 Unauthorized` error.

(c) The request will return an empty response with a `200 OK` status.

(d) The request will fail with a `403 Forbidden` error.

[Source ans: b]

8. [Source Q32; MCQ]

What will be the result of the following fetch request.

```
fetch("http://127.0.0.1:5000/protected", {
        method: "GET",
        headers: {
            "Authorization": "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVJ9.."
        }
    })
    .then(response => {
        if (response.status === 403) {
            throw new Error("403 Forbidden: Insufficient permissions");
        }
        return response.json();
    })
    .then(data => console.log("Protected Route Response:", data))
    .catch(error => console.error("Error:", error));
```

(a) The request will be processed successfully, and "Hello, student!" will be returned.

(b) The request will fail with a `401 Unauthorized` error.

(c) The request will return an empty response with a `200 OK` status.

(d) The request will fail with a `403 Forbidden` error.

[Source ans: d]

Shared context (source Q33)

Consider the following Vue app with the Vue Router shown below and answer the given sub-questions.

```
const Login = { template: "<p>You are not logged in.</p>" }
const Profile = { template: "<p>Welcome to profile</p>" }
const Unauthorized = { template: "<p>You are not authorized</p>" }

const router = new VueRouter({
    routes: [
        { path: '/login', component: Login },
        { path: '/unauthorized', component: Unauthorized },
        { path: '/profile', component: Profile }
    ]
})

new Vue({
    el: '#app',
    router,
    data:{
        userRole: '',
        isAuthenticated: false
    },
    created() {
        this.checkAuthentication();
    },
    methods: {
        checkAuthentication() {
            const token = localStorage.getItem('token');
            console.log(`token got: ${token}`)
            this.isAuthenticated = !!token;
            // this.isAuthenticated = token ? true : false
            if (this.isAuthenticated) {
                this.userRole = localStorage.getItem('userRole') || 'user';
                console.log(`role got: ${this.userRole}`)
            } else {
                console.log("no token got")
                this.$router.replace('/login');
            }
        },
```

```
        hasAccess(requiredRole) {
            const roleHierarchy = {
                'admin': 3,
                'manager': 2,
                'user': 1
                };
            const currentRoleLevel = roleHierarchy[this.userRole] || 0;
            const requiredRoleLevel = roleHierarchy[requiredRole] || 0;
            console.log(`current role: ${this.userRole}`)
            console.log(`required role: ${requiredRole}`)

            return currentRoleLevel >= requiredRoleLevel;
        }
    },
    mounted() {
        if (!this.hasAccess('manager')) {
            console.log("has manager access")
            this.$router.replace('/unauthorized');
        }
        else{
            console.log(`has ${this.userRole} access`)
            this.$router.replace('/profile')
        }
    }
})
```

9. [Source Q34; MCQ]

What will be rendered on the browser if the application is loaded on the browser for the first time?

(a) You are not logged in.

(b) Welcome to profile

(c) You are not authorized.

(d) None of these.

[Source ans: c]

10. [Source Q35; MCQ]

What will be rendered on the browser for the given value of localStorage?

```
{
    token: 112,
    userRole: "manager"
}
```

(a) You are not logged in.

(b) Welcome to profile

(c) You are not authorized.

(d) None of these.

[Source ans: b]

Shiva(25f1000035)

2025 Aug31 ET

11. [Source Q6; MSQ]

Which of the following practices help protect against supply chain attacks?

(a) Storing keys only in environment variables

(b) Version pinning

(c) Using only packages with over 1000 stars on GitHub

(d) Reduce Dependencies

[Source ans: b, d]

2025 Dec18 ET

12. [Source Q4; MCQ]

Which of the following is considered a good practice for securing user authentication in a web application?

(a) Storing passwords as plain text in the database for quick lookup

(b) Using hashing algorithms like ”bcrypt” or ”Argon2” to store passwords securely

(c) Including user passwords directly in SQL queries for verification

(d) All of these

[Source ans: b]

13. [Source Q5; MCQ]

In the following code snippet, which values should replace option1 and option2 so that the cookie is sent only over HTTPS and is restricted from cross-site requests?

```
response.set_cookie('session_id', 'abc123', secure=option1,
samesite=option2)
```

(a) secure=True and samesite=’Strict’

(b) secure=False and samesite=’Strict’

(c) secure=True and samesite=’None’

(d) secure=False and samesite=’Lax’

[Source ans: a]

14. [Source Q8; MCQ]

How can you stop a Cross-Site Scripting (XSS) attack?

(a) Backend validation of user input and make sure any user-provided text is not executed as code.

(b) Store authentication token or jwt in cookies.

(c) Use v-html to show user generated data.

(d) Send cookies with every request to the backend.

[Source ans: a]

2026 Apr06 Quiz-2

15. [Source Q1; MCQ]

You are building a login feature for a web application. The frontend sends the user's email and password to the backend API. The backend validates credentials against a database. Which security issues exist in the code given below? Frontend: script.js

```
methods: {
  async login() {
    const response = await fetch('/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email: this.email, password: this.password })
    });
    const data = await response.json();
    console.log(data.message);
  }
}
```

Backend: app.py

```
@app.route('/login', methods=['POST'])
def login():
    email = request.json['email']
    password = request.json['password']
    query = f"SELECT * FROM users WHERE email = '{email}'
            AND password = '{password}'"
    result = db.execute(query)
    if result:
        return jsonify({"message": "Login successful"})
    return jsonify({"message": "Invalid credentials"})
```

(a) The SQL query is vulnerable to SQL injection attacks

(b) Passwords are stored and compared in plain text

(c) No authentication token (session/JWT) mechanism is implemented

(d) All of these.

[Source ans: d]

16. [Source Q14; MSQ]

Consider a scenario involving localStorage, Promise chains, and Vue lifecycle hooks: Scenario: A user authentication system where: On app initialization, the app checks localStorage for a stored user token• If a token exists, it validates the token asynchronously (via a Promise)• Based on validation, different UI components are rendered• The user navigates between different views• Code in mounted() hook:

```
mounted() {
    const storedToken = localStorage.getItem('userToken');
    if (storedToken) {
        this.validateToken(storedToken)
            .then(user => {
                this.user = user;
                this.isAuthenticated = true;
            })
            .catch(err => {
                localStorage.removeItem('userToken');
                this.isAuthenticated = false;
            });
    }
}
```

Which of the following statements is/are CORRECT about this implementation?

(a) If the page is refreshed while the Promise is pending, the token will be lost because localStorage is cleared on refresh

(b) If token validation fails, the token is removed from storage, preventing automatic login on future page loads

(c) The isAuthenticated flag might be false even though a storedToken exists, until the Promise validation completes

(d) sessionStorage should be used instead of localStorage because user tokens should not persist across browser sessions

(e) If the user navigates to a different route while validation is pending, the Promise will still complete and update the user data

[Source ans: b, c, e]

2026 May06 ET

17. [Source Q3; MCQ]

What is the main advantage of using JWT over session-based authentication?

(a) JWTs require server-side storage for each session.

(b) JWTs are stateless and eliminate the need for server-side session storage.

(c) JWTs are longer and contain more data than session IDs.

(d) JWTs automatically handle token expiration without additional logic.

[Source ans: b]

18. [Source Q6; MCQ]

A JWT (JSON Web Token) is represented in the format:

`"header.payload.signature"`

.In authentication using Flask-JWT-Extended, what does the payload part of the JWT stores?

(a) Encryption keys used to secure the token

(b) User-related data (claims) such as identity and expiration time

(c) The algorithm used for signing the token

(d) The hashed signature of the token

[Source ans: b]

19. [Source Q19; MCQ]

Consider the following endpoints created with a working setup of flask-security and flask-sqlalchemy.

```
@app.route('/api/open')
def open_endpoint():
   return jsonify({'message': 'Open access', 'status': 'success'})

@app.route('/api/secure')
@auth_required('token')
def secure_endpoint():
   return jsonify({'message': 'Secure access', 'status': 'authenticated'})

@app.route('/api/superuser')
@auth_required('token')
@roles_required('superuser')
def superuser_endpoint():
   return jsonify({'message': 'Superuser access', 'status': 'authorized'})
```

Three requests are made to this application: GET /api/open (no authentication headers)1. GET /api/secure (no authentication headers)2. GET /api/superuser with valid authentication token but user has role 'basic' (not 'superuser')3. What will be the response status codes for these requests, respectively?

(a) 200, 200, 200

(b) 200, 401, 403

(c) 200, 403, 401

(d) 401, 403, 403

[Source ans: b]

20. [Source Q24; MSQ]

Which of the following statement(s) is/are true about GraphQL?

(a) Mutations in GraphQL can be used to change the underlying data store.

(b) GraphQL services are created by defining types and fields of those types, then providing functions for each field.

(c) GraphQL can only fetch data from a single source.

(d) GraphQL allows only GET operations.

[Source ans: a, b]

21. [Source Q25; MSQ]

Consider the following code:

```
from flask import Flask, jsonify, request
from flask_jwt_extended import JWTManager, create_access_token,
jwt_required, get_jwt_identity

app = Flask(__name__)
app.config["JWT_SECRET_KEY"] = "secret"
jwt = JWTManager(app)

@app.route("/login", methods=["POST"])
def login():
    username = request.json.get("username")
    access_token = create_access_token(identity=username)
    return jsonify(access_token=access_token)

@app.route("/stats")
@jwt_required()
def stats():
    current_user = get_jwt_identity()
    return jsonify(message=f"Welcome {current_user}")
```

You are developing an analytics platform using Flask and Flask-JWT-Extended. A user sends a POST request to `/login` with:

`{ "username": "Jassi" }`

. Then tries to access `/stats` . Which of the following statements are correct?

(a) The `/stats` route will return ”Welcome Jassi” only if a valid JWT token is sent in the Authorization header

(b) If no token is sent, accessing `/stats` will result in an unauthorized error

(c) The JWT token must be sent as JSON in the request body to access `/stats`

(d) None of these

[Source ans: a, b]

2026 Aug14 Quiz-2

22. [Source Q9; MSQ]

A Vue frontend, a mobile app, and another backend service all call the same Flask API deployed on multiple servers. The team chooses JWT authentication. Which of the following statements are correct?

(a) Each request can carry the token, so any server can validate it without storing a per-user session in memory.

(b) JWT is API-friendly because clients can send it using the

```
Authorization: Bearer <token>
```

header.

(c) In Flask-JWT-Extended, `get_jwt_identity()` can read the identity stored in the token.

(d) Session-based authentication is usually easier to scale across multiple servers because no shared storage is needed. The team should have chosen this.

[Source ans: a, b, c]

2026 Sep13 ET

23. [Source Q1; MCQ]

A JWT token contains three parts:

`header.payload.signature`

. Which of the following best explains the relationship between the header and the signature in JWT authentication?

(a) The header stores the encrypted password used to generate the signature.

(b) The signature only protects the payload section; changes to the header do not affect token validation.

(c) The signature is generated using the encoded header, encoded payload, and a secret key. If the header is modified, the signature verification fails.

(d) The header contains the final signature hash generated after token creation.

[Source ans: c]

24. [Source Q19; MSQ]

Consider the following Flask JWT code:

```python
from flask import Flask, jsonify
from flask_jwt_extended import JWTManager, create_access_token,
jwt_required

app = Flask(__name__)
app.config["JWT_SECRET_KEY"] = "secret"
jwt = JWTManager(app)

@app.route("/token")
def token():
    access_token = create_access_token(identity="Ravi")
    return jsonify(access_token=access_token)

@app.route("/dashboard")
@jwt_required()
def dashboard():
    return jsonify(message="Dashboard Access Granted")
```

A client first accesses `/token` and then tries to access `/dashboard` . Which of the following statements are correct?

(a) `/dashboard` can only be accessed if a valid JWT token is sent with the request.

(b) If the JWT token is missing or invalid, Flask-JWT-Extended returns an unauthorized error.

(c) The JWT token is typically sent in the `Authorization` header while accessing protected routes.

(d) The token generated by `/token` is automatically stored by Flask and reused for future requests.

[Source ans: a, b, c]

25. [Source Q21; MSQ]

Consider the following code using flask_jwt_extended:

```python
app.config["JWT_SECRET_KEY"] = "my-secret"
jwt = JWTManager(app)

@jwt.user_identity_loader
def load(user):
    return user.username

@jwt.user_lookup_loader
def user_lookup(_jwt_header, jwt_data):
    identity = jwt_data["sub"]
    return User.query.filter_by(username=identity).one_or_none()

@app.route('/login', methods=["POST"])
def login():
    user = User.query.filter_by(username="u1").one_or_none()
    token = create_access_token(identity=user)
    return {"access_token": token}

@app.route('/dashboard')
@jwt_required()
def dashboard():
    return {
        "username": current_user.username
    }
```

Which of the following statements are correct? (Select all that apply)

(a) The `@jwt_required()` decorator ensures that only requests with a valid JWT can access `/dashboard` .

(b) The `user_identity_loader` determines what value is stored in the ”sub” field of the JWT.

(c) If `user_lookup_loader` is removed, `current_user` will not be automatically populated.

(d) The

`create_access_token(identity=user)`

directly stores the entire user object inside the JWT.

(e) If no JWT is sent in the request, `/dashboard` will still execute but `current_user` will be None.

(f) The `current_user` object is reconstructed using the identity extracted from the JWT.

[Source ans: a, b, c, f]